

1. Access Modifiers: **private** vs **public**

1.1 **private**

The **private** access modifier restricts access to members (fields or methods) within the same class only. It is primarily used to enforce the principle of **encapsulation**, which ensures that the internal state of an object cannot be modified directly from outside the class.

Example:

```
class BankAccount {
    private double balance = 1000;

    public double getBalance() {
        return balance;
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount();
        System.out.println(acc.getBalance()); // OK - accessing through public
        // System.out.println(acc.balance);    // ERROR - balance is private
    }
}
```

⇒ Here, **balance** is only visible inside **BankAccount**. Any other class that tries to access **acc.balance** directly will get a compile-time error. The only way to read it is through **getBalance()**.

Key Points:

- Prevents direct access from outside the class
- Ensures controlled modification via methods
- Commonly used for class fields
- Errors are caught at compile-time, not runtime

1.2 **public**

The **public** access modifier allows unrestricted access from any other class, in any package. It is typically used to expose methods that define the **interface (API)** of a class.

Example:

```

class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println(calc.add(3, 5)); // OK - add() is public
    }
}

```

⇒ `add()` is the public **API** (the set of methods exposed for external use) of the `Calculator` class. Note: "API" here does not refer to the Java `interface` keyword — it means the publicly available methods that other classes can call.

Key Points:

- Accessible from any package
- Used for methods intended for external use

1.3 Best Practice

- Fields (attributes) should be declared `private`
- Public methods should be used to interact with the object's state
- Internal helper methods that only serve the class's own logic should also be `private`

```

class BankAccount {
    private double balance;

    public void withdraw(double amount) {
        if (isValidAmount(amount)) {
            balance -= amount;
        }
    }

    // Helper method - only used internally, no reason to expose
    private boolean isValidAmount(double amount) {
        return amount > 0 && amount <= balance;
    }
}

```

⇒ `isValidAmount` is internal logic. No other class needs to call it directly, so it stays `private`.

2. static vs non-static

2.1 static Members

Static members belong to the **class** rather than any instance. Only **one copy** exists in memory (stored in the Method Area/Metaspace of the JVM), shared among all objects.

Example:

```
class Counter {
    static int count = 0;

    Counter() {
        count++;
    }

    public static int getCount() {
        return count;
    }
}

public class Main {
    public static void main(String[] args) {
        Counter a = new Counter(); // count = 1
        Counter b = new Counter(); // count = 2
        Counter c = new Counter(); // count = 3

        // Access via class name – no object needed
        System.out.println(Counter.getCount()); // 3
    }
}
```

⇒ Three objects were created, but `count` has only one copy. All three constructors incremented the same variable.

Key Points:

- Accessed via class name: `ClassName.method()`
- No object instantiation required
- Suitable for utility methods or shared data

2.2 Non-static Members

Non-static members (instance members) belong to a specific object. Each object maintains its own independent copy.

Example:

```
class Student {
    int score = 0; // non-static - each object has its own copy
}

public class Main {
    public static void main(String[] args) {
        Student a = new Student();
        Student b = new Student();
        a.score = 90;

        System.out.println(a.score); // 90
        System.out.println(b.score); // 0 - not affected by a.score
    }
}
```

⇒ Changing `a.score` has no effect on `b.score`. They are independent copies.

Key Points:

- Requires object instantiation
- Each object has independent state

2.3 Comparison

Feature	static	non-static
Ownership	Class	Object
Memory	Shared	Separate
Access	ClassName.method()	object.method()

2.4 Important Restriction

A `static` method **cannot** access non-static members directly, because when the static method runs, no object may exist yet.

```

class Example {
    int x = 10; // non-static

    public static void main(String[] args) {
        // System.out.println(x); // ERROR - main is static, x is non-static

        Example obj = new Example();
        System.out.println(obj.x); // OK - accessing through an object
    }
}

```

3. Access Modifiers and Inheritance

3.1 Types of Access Modifiers

- **private**: accessible only within the same class
- default (no modifier): accessible within the same package
- **protected**: accessible within package and subclasses
- **public**: accessible everywhere

3.2 Rules in Inheritance

The overriding rule in Java: when overriding a method, the access level can stay the same or become less restrictive. It can never become more restrictive.

In other words: **private** < **default** < **protected** < **public**. You can only move right (wider), never left (narrower).

Rule 1: Public Methods

A method declared as **public** in a superclass **must** remain **public** in subclasses. There is no wider level available, so it stays **public**.

```

class Animal {
    public void speak() {
        System.out.println("Animal speaks");
    }
}

class Dog extends Animal {
    @Override
    public void speak() { // Must stay public
        System.out.println("Dog barks");
    }
}

```

Rule 2: Protected Methods

A method declared as `protected` can be overridden as `protected` or `public`, but **not** `default` or `private`.

```

class Animal {
    protected void move() {
        System.out.println("Animal moves");
    }
}

class Dog extends Animal {
    @Override
    public void move() { // OK – widened from protected to public
        System.out.println("Dog runs");
    }
}

```

Rule 3: Default Methods

A method with default access (no modifier) can be overridden as `default`, `protected`, or `public`. It **cannot** be overridden as `private` — that would reduce visibility.

```
class Animal {
    void rest() { // default access
        System.out.println("Animal rests");
    }
}

class Dog extends Animal {
    @Override
    protected void rest() { // OK – widened from default to protected
        System.out.println("Dog sleeps");
    }

    // private void rest() { } // ERROR – cannot reduce visibility
}
```

Rule 4: Private Methods

Private methods are **not inherited** and **cannot be overridden**. If a subclass declares a method with the same name as a **private** method in the superclass, it is a completely new method — Java does not treat them as related.

```

class Parent {
    private void secret() {
        System.out.println("Parent secret");
    }

    public void callSecret() {
        secret(); // calls Parent's own private method
    }
}

class Child extends Parent {
    // This is NOT an override. It is a brand new method.
    void secret() {
        System.out.println("Child secret");
    }
}

public class Main {
    public static void main(String[] args) {
        Child c = new Child();
        c.callSecret(); // prints "Parent secret" - Parent's private method
        c.secret();     // prints "Child secret" - Child's own method
    }
}

```

⇒ `callSecret()` is inherited and still calls `Parent`'s private `secret()`, not `Child`'s. This proves that `Child.secret()` is not an override.

4. General Principles

4.1 Do Not Reduce Visibility

A subclass **cannot** override a method with a more restrictive access level. This is enforced by the compiler.

Allowed direction: `default` → `protected` → `public` (widening). Blocked direction: `public` → `protected` → `default` → `private` (narrowing).

4.2 Encapsulation First

Fields should be **private**. Access and modification should go through **public** getter and setter methods. Setter methods are where you put validation logic.

```
class Student {
    private int age;

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        if (age > 0 && age < 150) {
            this.age = age;
        }
    }
}
```

4.3 Static Methods Are Hidden, Not Overridden

When a subclass declares a **static** method with the same signature as a **static** method in the superclass, this is called **method hiding**, not overriding.

The difference: with overriding (non-static), Java picks the method based on the **actual object type** at runtime. With hiding (static), Java picks the method based on the **declared variable type** at compile-time.

```

class Parent {
    static void greet() { System.out.println("Parent"); }
}

class Child extends Parent {
    static void greet() { System.out.println("Child"); } // hiding, not over
}

public class Main {
    public static void main(String[] args) {
        Parent obj = new Child();
        obj.greet(); // prints "Parent" — because declared type is Parent

        Child obj2 = new Child();
        obj2.greet(); // prints "Child" — because declared type is Child
    }
}

```

⇒ If `greet()` were non-static (and properly overridden), `obj.greet()` would print "Child" due to dynamic dispatch. But since it is static, the declared type (`Parent`) determines which version runs.

5. Summary

- `private` protects internal data — accessible only within the same class
- `public` defines the external interface — accessible from anywhere
- `static` members belong to the class, shared across all instances
- Non-static members belong to individual objects, each with an independent state
- When overriding, access level can only stay the same or widen — never narrow
- `private` methods are not inherited and cannot be overridden
- `static` methods are hidden, not overridden — dispatch is based on declared type, not actual object type